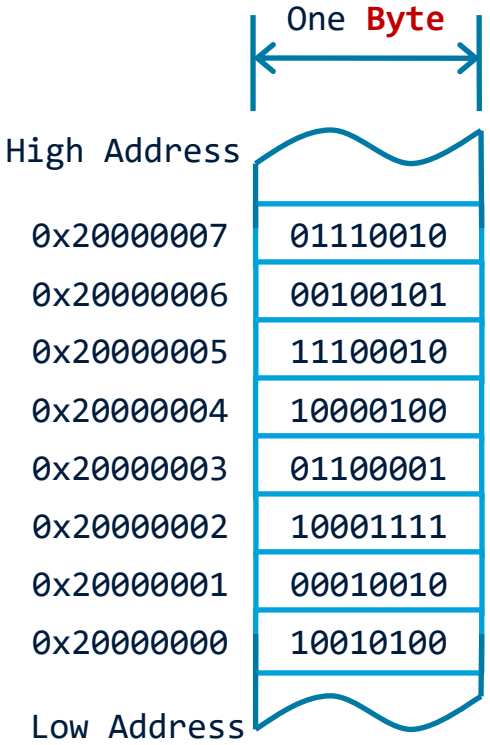
Embedded Programming

C Pointers

Dr. Charis Kouzinopoulos
Department of Advanced Computing Sciences
Maastricht University
2024-2025
charis.kouzinopoulos@maastrichtuniversity.nl

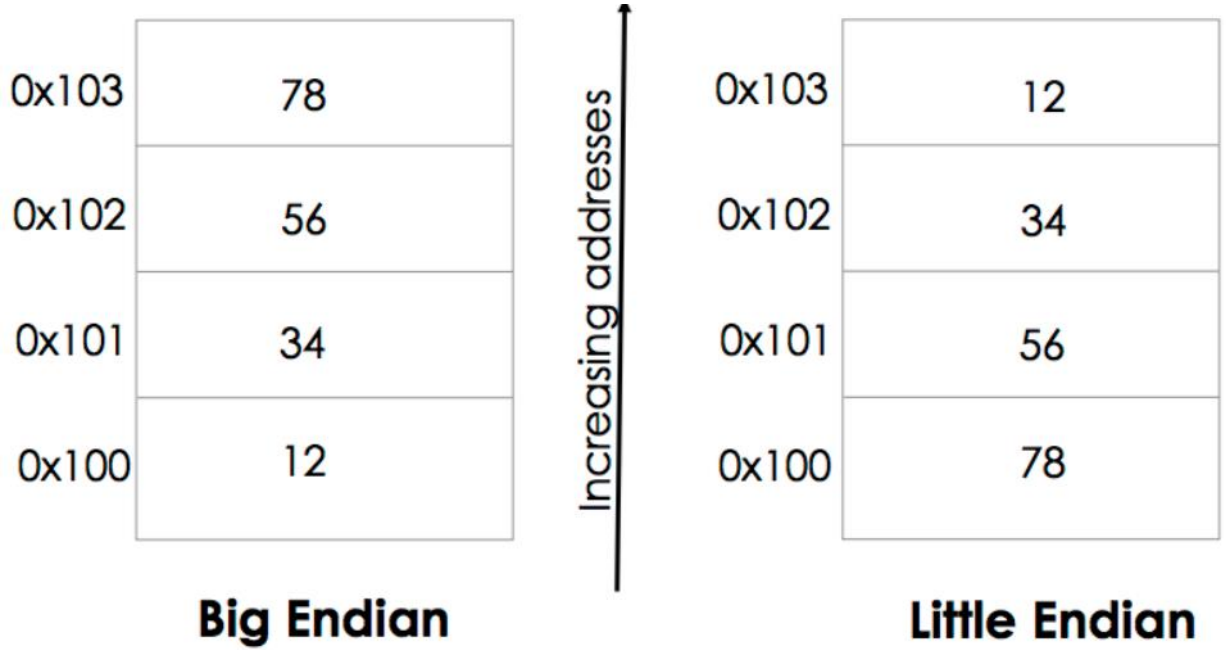
Memory Address

- Memory is byte addressable
  - Memory is an array of bytes
  - Each byte has a memory address
  - Smallest data that can be addressed is a byte
  - Each address has 32 bits (ARM Cortex-M)
- An object may occupy multiple bytes
  - A word has four bytes
  - Word: Little endian *vs* Big endian



Big Endian vs. Little Endian

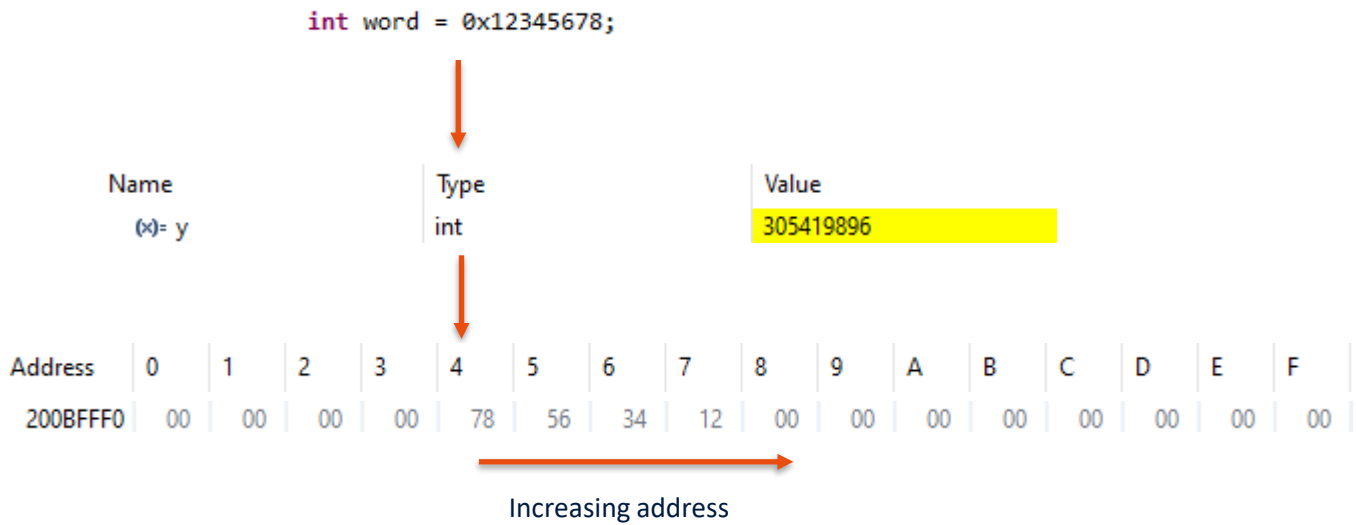
- How is a word, say, 0x12345678 stored in memory?



Big Endian vs. Little Endian

LIVECODE

- Lets try it out in the IDE:



Big Endian vs. Little Endian

- **Big-endian (big end first)**
  - Most significant byte of any multi-byte data field is stored at the lowest memory address
  - Reading from left to right
  - SPARC & Motorola
- **Little-endian (little end first)**
  - Least significant byte of any multi-byte data field is stored at the lowest memory address
  - Reading from right to left instead
  - Intel processors
- **Bi-endian (ARM, PowerPC, Alpha)**
  - Can be configured to view words stored in memory as either Big-endian or Little-Endian Format

*In Cortex-M33, bytes are coded in memory by default in **Little Endian** format. The lowest numbered byte in a word is considered the word's least significant byte and the highest numbered byte the most significant*

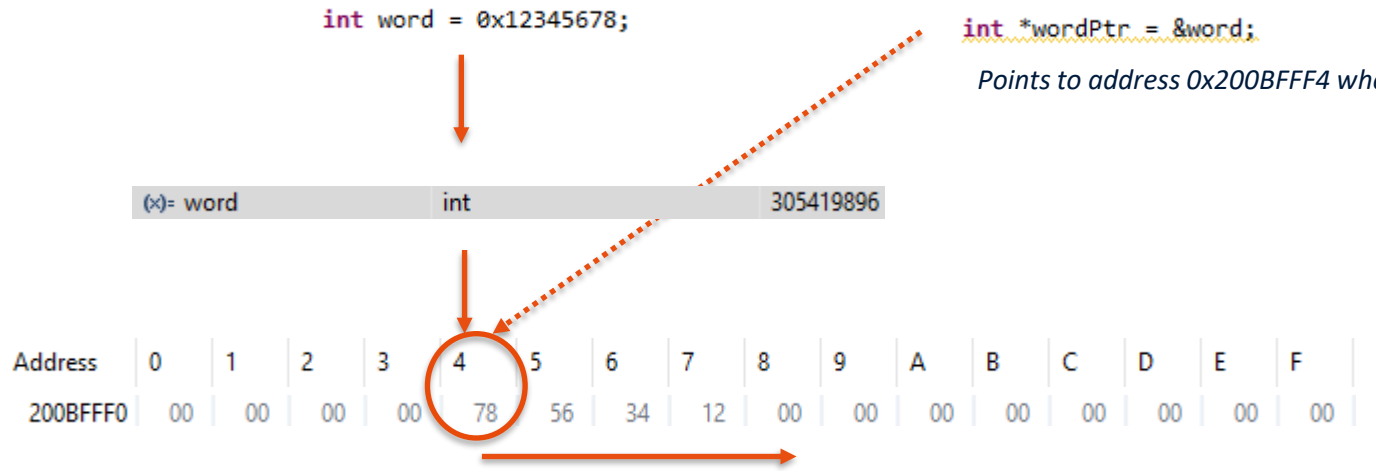
Pointers

- A pointer is a variable that stores the **memory address** of another variable. Instead of holding a data value directly, a pointer "points" to the location in memory where the data is stored.

Pointers

- A pointer is a variable that stores the **memory address** of another variable. Instead of holding a data value directly, a pointer "points" to the location in memory where the data is stored.

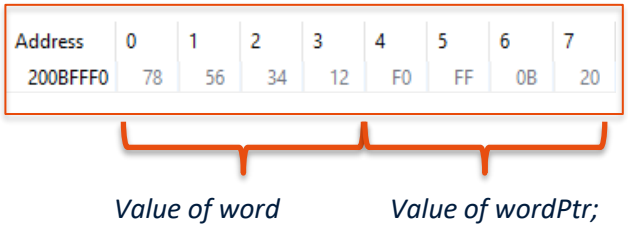
Address	0	1	2	3	4	5	6	7
200BFFF0	78	56	34	12	F0	FF	0B	20



Pointers

LIVECODE

- A pointer is a variable that stores the **memory address** of another variable. Instead of holding a data value directly, a pointer "points" to the location in memory where the data is stored.



Pointers

```
int a = 5;
int *ptr = 0;
```

Address	Value
0x8130	0x00000005
0x8134	0x00000000

```
ptr = &a;
```

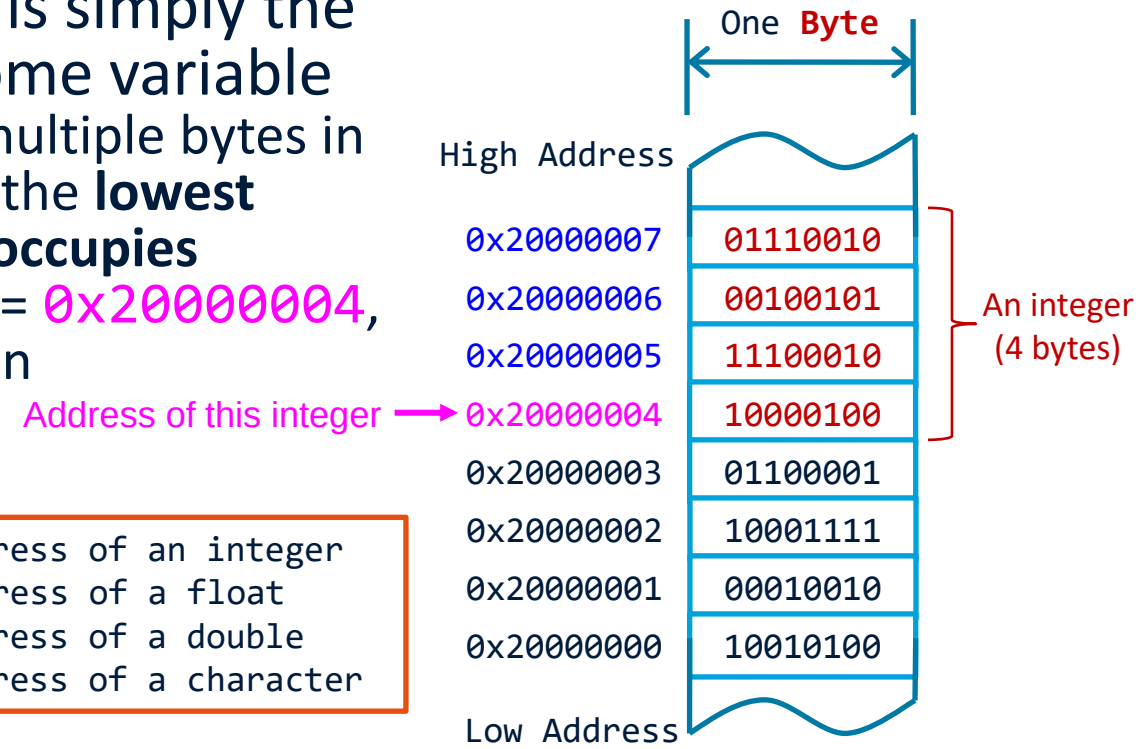
Address	Value
0x8130	0x00000005
0x8134	0x00008130

```
*ptr = 8;
```

Address	Value
0x8130	0x00000008
0x8134	0x00008130

Pointers

- The value of a *pointer* is simply the memory address of some variable
  - If a variable occupies multiple bytes in memory, its address is the **lowest address of all bytes it occupies**
  - Address of the integer = `0x20000004`, regardless of the endian



- Define pointers

```
int *ip;    // ip can hold the address of an integer
float *fp;  // fp can hold the address of a float
double *dp; // dp can hold the address of a double
char *cp;   // cp can hold the address of a character
```

Reference and Dereference Operators

Reference Operator (&)

- Expression &x returns the address of variable x
- Ampersand (&) is called *reference operator* or *address-of operator*

Dereference Operator (*)

- Expression *p returns the value of the variable to which p points
- The asterisk (*) is called *dereference operator*

```
int x = 1;
```

Reference and Dereference Operators

Reference Operator (&)

- Expression &x returns the address of variable x
- Ampersand (&) is called *reference operator* or *address-of operator*

Dereference Operator (*)

- Expression *p returns the value of the variable to which p points
- The asterisk (*) is called *dereference operator*

```
int x = 1;
int *ptr;  // Define a pointer that
           // can point to an integer
```

memory

Reference and Dereference Operators

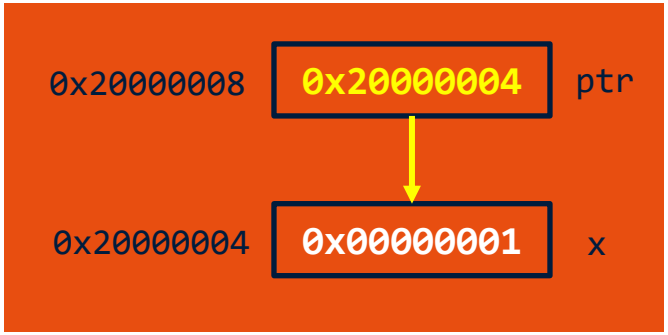
Reference Operator (&)

- Expression &x returns the address of variable x
- Ampersand (&) is called *reference operator* or *address-of operator*

Dereference Operator (*)

- Expression *p returns the value of the variable to which p points
- The asterisk (*) is called *dereference operator*

```
int x = 1;
int *ptr;  // Define a pointer that
           // can point to an integer
ptr = &x;  // Make the pointer points
           // to variable x
```



memory

Reference and Dereference Operators

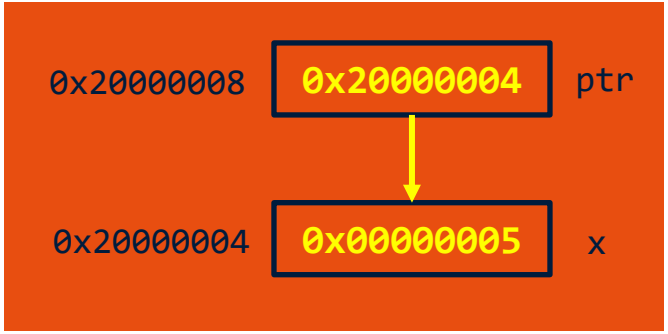
Reference Operator (&)

- Expression &x returns the address of variable x
- Ampersand (&) is called *reference operator* or *address-of operator*

Dereference Operator (*)

- Expression *p returns the value of the variable to which p points
- The asterisk (*) is called *dereference operator*

```
int x = 1;
int *ptr;  // Define a pointer that
           // can point to an integer
ptr = &x;  // Make the pointer points
           // to variable x
*ptr = 5;  // Dereferencing
```



Reference and Dereference Operators

Reference Operator (&)

- Expression &x returns the address of variable x
- Ampersand (&) is called *reference operator* or *address-of operator*

Dereference Operator (*)

- Expression *p returns the value of the variable to which p points
- The asterisk (*) is called *dereference operator*

Summary:

```
&var:  address of var
*ptr:  value at address pointed by ptr
```

Pointers examples

// Create an int variable and a pointer to point to the variable's address
int myvar = 42;
int mypointer = &myvar;*
// Now that mypointer contains the **memory address** of myvar, it can be used to access the **value** stored in myvar through the use of the * operator
*int xx = *mypointer;*

Size of a pointer

int size1 = sizeof(int);*
int size2 = sizeof(double);*

size1	int	4
size2	int	4

Pointers are typically 4 bytes (32 bits) on 32-bit systems. So, what is then the purpose of specifying a type while declaring a pointer?

In C, the type of a pointer is important because it helps the compiler check for errors and generate correct code for accessing and manipulating the value the pointer points to. This is related to how the data is stored in memory. In modern architectures, each memory address refers to a cell of memory that is 8 bits (1 byte) in size

Pointers examples

*char *address = (char *) 0x20000000;*

*char data = *address;*

What is the above code doing?
How many bytes of data are read?

Pointers examples

*char *address = (char *) 0x20000000;*

*char data = *address;*

What is the above code doing?
How many bytes of data are read?
1 byte of data is read from memory location 0x20000000. Why 1 byte? Because the pointer points at char data

Pointers examples

*char *address = (char *) 0x20000000;*

**address = 0x89;*

What is the above code doing?

Pointers examples

*char *address = (char *) 0x20000000;*

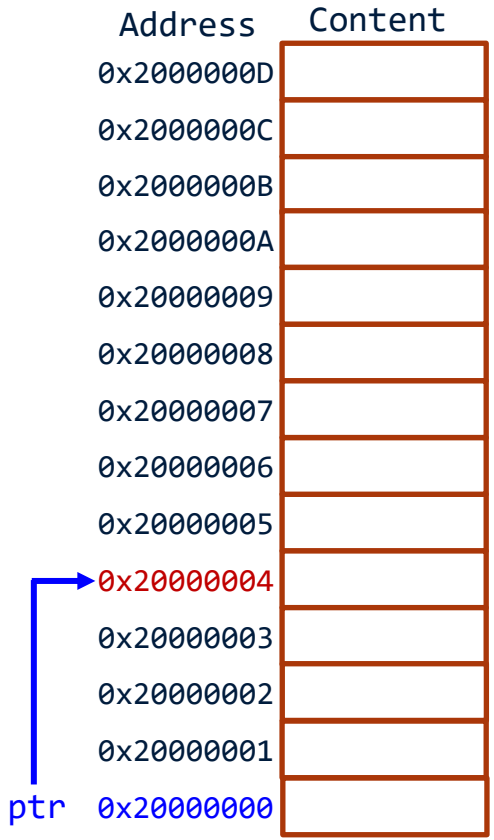
**address = 0x89;*

What is the above code doing?

The address pointer is dereferenced to write data to memory location 0x20000000

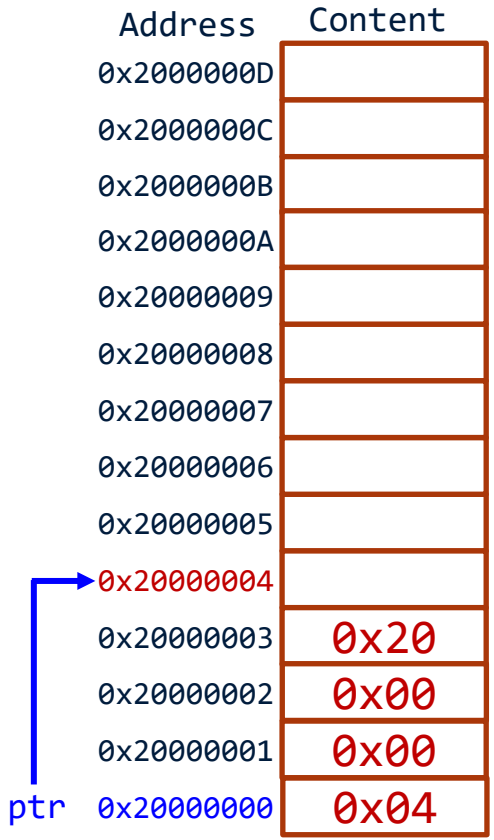
Pointer example

- Define a pointer *ptr*, pointing to memory address 0x20000004
- How would this pointer store its data in memory in little endian systems?



Pointer Arithmetic

- Pointer arithmetic is a powerful feature that allows to perform operations on pointers, such as incrementing, decrementing, and calculating the difference between pointers
- These operations are particularly useful when working with arrays or managing dynamic memory

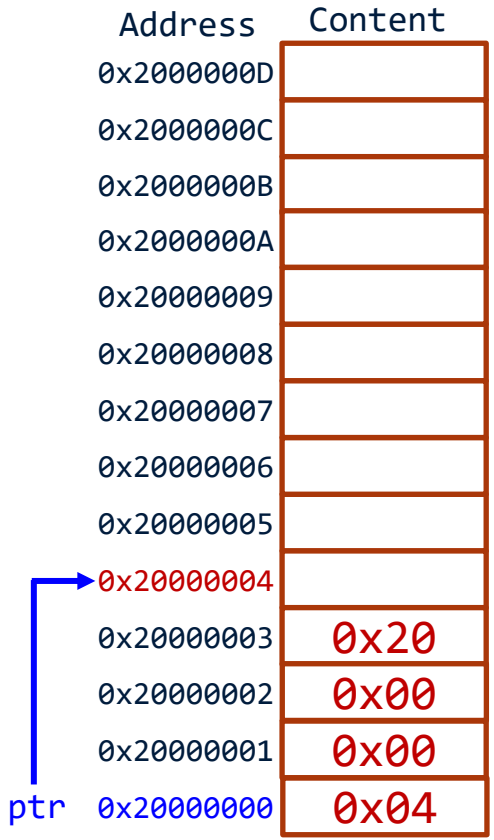


Pointer Arithmetic

- ptr is pointer, and stored at `0x20000000`
- `ptr = 0x20000004`
- What happens if:

```
ptr++;
```

- Is it true? `ptr = 0x20000005`



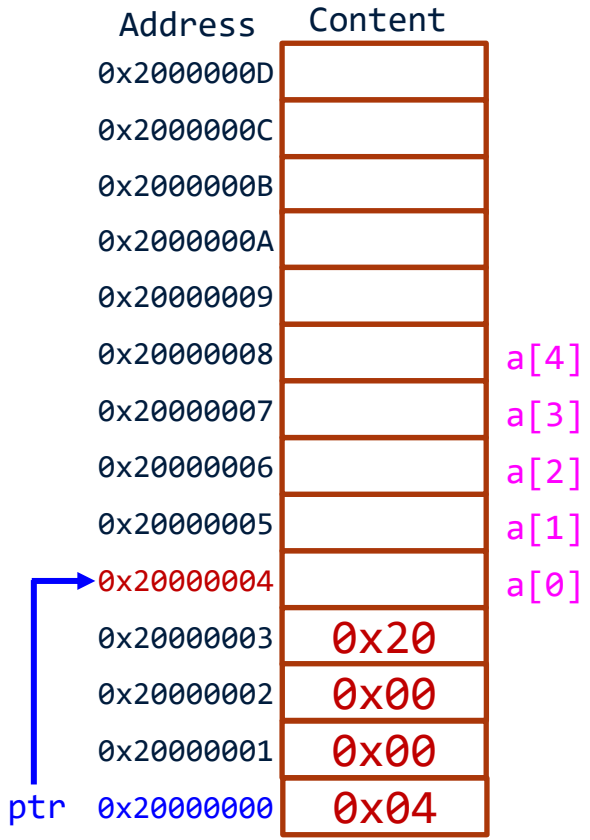
Pointer Arithmetic

- ptr is pointer, and stored at `0x20000000`
- `ptr = 0x20000004`
- What happens if:

```
ptr++;
```

- Is it true? `ptr = 0x20000005`

```
char *ptr;
char a[5];
ptr = &a[0];
ptr++;
```



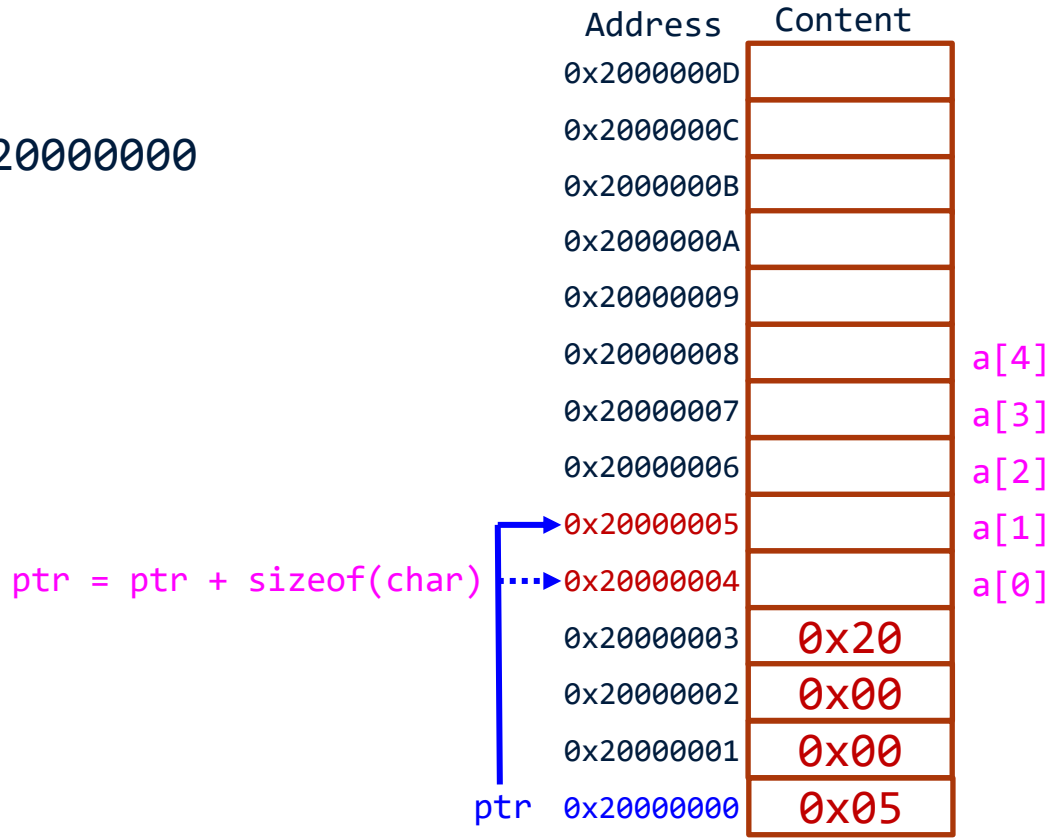
Pointer Arithmetic

- ptr is pointer, and stored at `0x20000000`
- `ptr = 0x20000004`
- What happens if:

```
ptr++;
```

- Is it true? `ptr = 0x20000005`

```
char *ptr;
char a[5];
ptr = &a[0];
ptr ++;
```



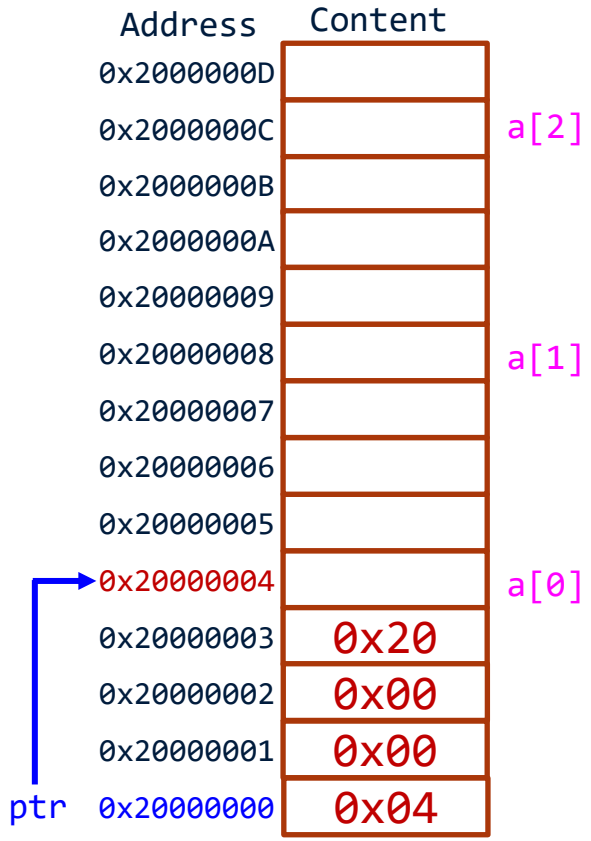
Pointer Arithmetic

- ptr is pointer, and stored at `0x20000000`
- `ptr = 0x20000004`
- What happens if:

```
ptr++;
```

- Is it true? `ptr = 0x20000005`

```
int *ptr;
int a[5];
ptr = &a[0];
ptr++;
```



Pointer Arithmetic

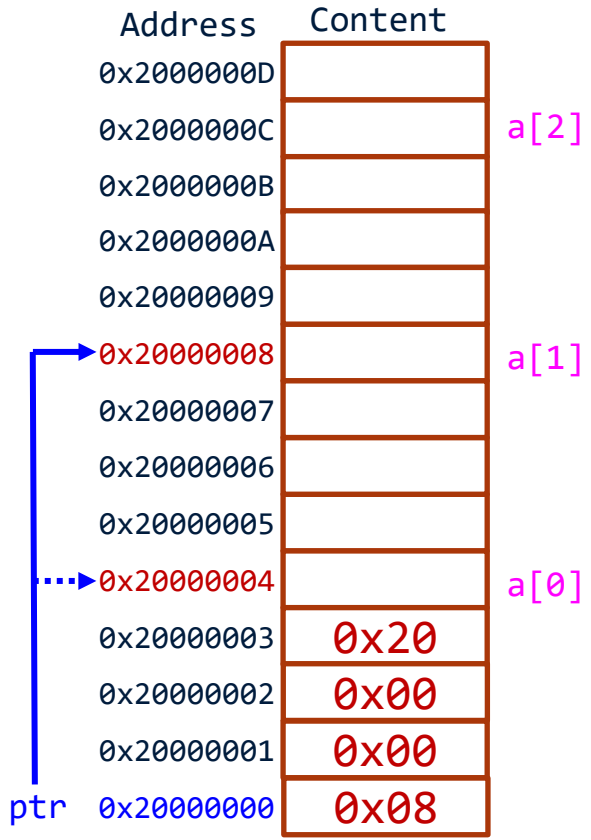
- ptr is pointer, and stored at `0x20000000`
- `ptr = 0x20000004`
- What happens if:

```
ptr++;
```

- Is it true? `ptr = 0x20000005`

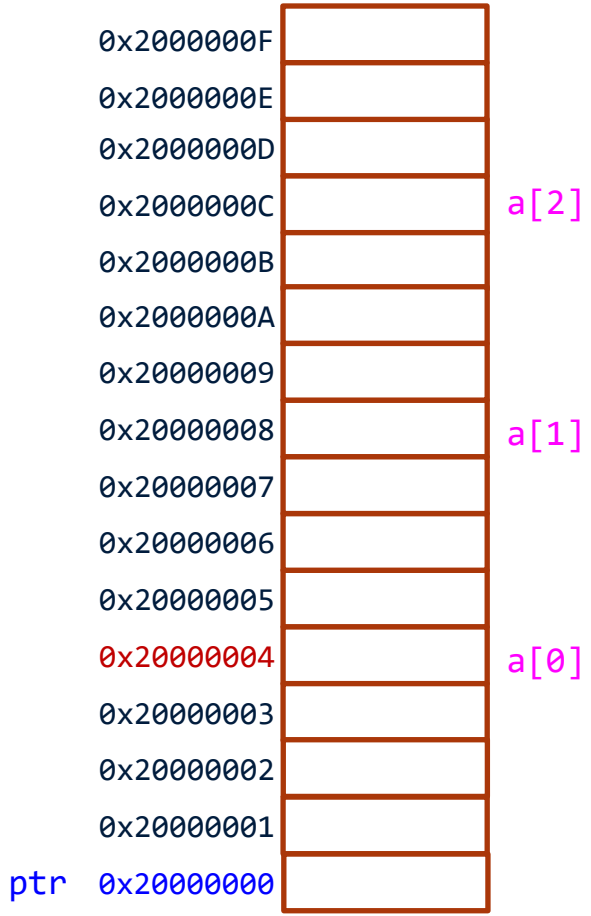
```
int *ptr;
int a[5];
ptr = &a[0];
ptr++;
```

`ptr = ptr + sizeof(int)`



Pointer and Array

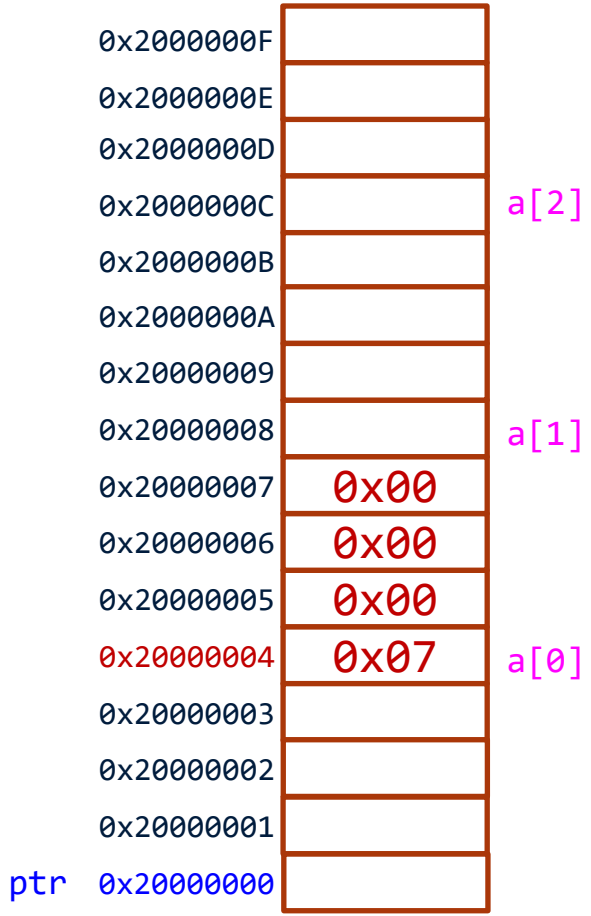
```
int *ptr;  // Assume stored at 0x20000000
int a[5];  // Assume started at 0x20000004
```



Pointer and Array

```
int *ptr;  // Assume stored at 0x20000000
int a[5];  // Assume started at 0x20000004
*a = 7;    // array[0] = 7;
```

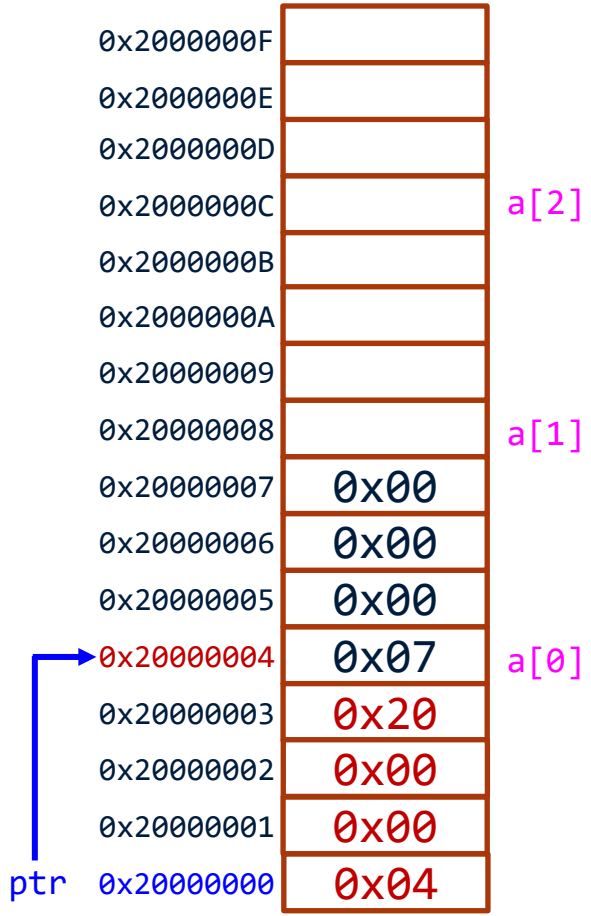
Array name is a pointer to the first element in the array!



Pointer and Array

```
int *ptr;  // Assume stored at 0x20000000
int a[5];  // Assume started at 0x20000004
*a = 7;    // array[0] = 7;
ptr = a;   // ptr points to array[0]
```

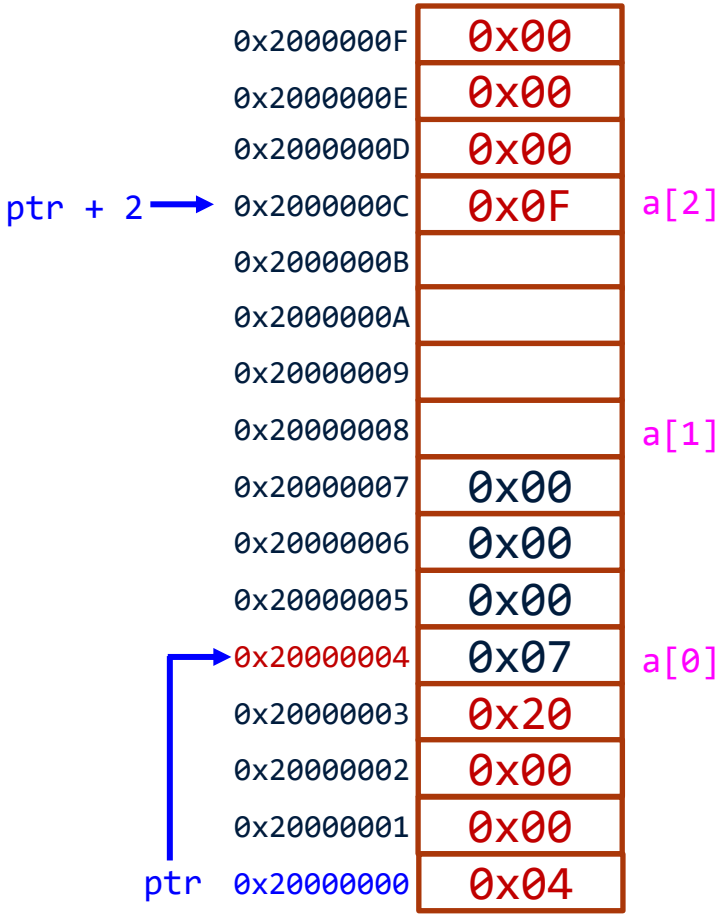
Array name is a pointer to the first element in the array!



Pointer and Array

```
int *ptr;  // Assume stored at 0x20000000
int a[5];  // Assume started at 0x20000004
*a = 7;    // array[0] = 7;
ptr = a;   // ptr points to array[0]
*(ptr + 2) = 15; // array[2] = 15
```

```
ptr = ptr + 2 * sizeof (int)
```



Pointer and Array

Example:

```
int array[5];        ← Declares 5 contiguous integers
int *ptr = array;    ← Arrays can be used as pointers
ptr[0] = 1;          ← Pointers can be indexed with array syntax
*(array + 1) = 2;    ← Arrays can be dereferenced with pointer syntax
*(1 + array) = 3;    ← Pointer addition is commutative
```

Pointer and Array

Example: int array[5] = {2, 4, 3, 1, 5};

If array is located in memory starting at address 0x1000 on a 32-bit little-endian machine then memory will contain the following (values and addresses in hex):

200BFFA0	3419	5BAB	0200	0000	0400	0000	0300	0000
200BFFB0	0100	0000	0500	0000	0585	92D9	E836	BE17

	0	1	2	3
1000	2	0	0	0
1004	4	0	0	0
1008	3	0	0	0
100C	1	0	0	0
1010	5	0	0	0

Pointer and Array

Points to remember:

- *array* means 0x1000
- *array + 1* means 0x1004: "+ 1" means to add the size of 1 int, which is 4 bytes
- **array* means to dereference the contents of array. Considering the contents as a memory address (0x1000), look up the value at that location (0x0002)
- *array[i]* means element number i, 0-based, of array which is translated into *(array + i)*
- *array + i* is the memory location of the (i)th element of array, starting at i=0
- *(array + i)* takes that memory address and dereferences it to access the value

Operator Precedence

- ++/-- have greater precedence than *
  - `*ptr++`
    - Equivalent to `*(ptr++)`
    - write/read the value pointed, and then increment the pointer
    - `x = *ptr++;` is equivalent to:

```
x = *ptr;
ptr++;
```

  - `(*ptr)++`
    - The value pointed by ptr is incremented by 1.
    - `x = (*ptr)++;` is equivalent to

```
x = *ptr
*ptr = * ptr + 1;
```

Let’s practice together!

Perform arithmetic operations on number using pointers:

- For two integer variables, **num1=10** and **num2=20**, find their sum and difference *using pointers*

Let’s practice together!

Perform arithmetic operations on number using pointers:

- For two integer variables, **num1=10** and **num2=20**, find their sum and difference *using pointers*

```
int num1, num2, result;
int *ptr1, *ptr2;

ptr1 = &num1; // ptr1 stores the address of num1
ptr2 = &num2; // ptr2 stores the address of num2
result = *ptr1 + *ptr2;
result = *ptr1 - *ptr2;
```

Call by reference

```
int add(int *x, int *y)
{
    int z = *x + *y;
    return z;
}

int main()
{
    int a = 10, b = 20;
    int c = add(&a, &b);
}
```

2. Create two pointers to int to store the two addresses

3. Dereference the pointers to access their values

1. Pass the addresses of a and b as arguments to add()

Let’s practice together!

Swap two numbers **num1=10** and **num2=20** using call by reference:

Let’s practice together!

Swap two numbers **num1=10** and **num2=20** using call by reference:

- Copy the value of num1 to some temporary variable (temp)
- Copy the value of num2 to the first number
- Copy back the value of num1 from temp to num2

Let’s practice together!

Swap two numbers **num1=10** and **num2=20** using call by reference:

```
void swap(int *num1, int *num2)
{
   int temp;

   temp = *num1; // Copy the value of num1 to some temp variable

   *num1= *num2; // Copy the value of num2 to num1
   *num2= temp; // Copy the value of num1 stored in temp to num2
}

int num1 = 10, num2 = 20;
swap(&num1, &num2);
```

Example: pointers and memory locations

What does the following code do? What is going to be the value of *counter* and *p_int* at the end?

```
int counter = 0;

int main() {
        int *p_int;
        p_int = &counter;
        while (*p_int < 21) {
                (*p_int)++;
        }
}
```

Example: pointers and memory locations

What does the following code do? What is going to be the value of *counter* and *p_int* at the end?

```
int counter = 0;

int main() {
        int *p_int;
        p_int = &counter;
        while (*p_int < 21) {
                (*p_int)++;
        }
}
```

21

The memory address of counter

Example: pointers and memory locations

We can set any address as the value of p_int:

```
int counter = 0;

int main() {
        int *p_int;
        p_int = &counter;
        while (*p_int < 21) {
                (*p_int)++;
        }
}
p_int = 0x20000002; // A lot of times the compilers will complain about this
p_int = (int *)0x20000002; // Better to cast the hex value as a pointer to int
```